

MUTED: Machine Unlearning based on Trapdoors Equipped with a Distillator

Ko-Han Chiu^a, An-Ting Chao^a, Po-Wen Chi^{a,*}, and Ming-Hung Wang^b

^a*Department of Computer Science and Information Engineering, National Taiwan Normal University, 162, Section 1, Heping E. Rd., Taipei City, 106, Taiwan, Taiwan*

^b*Department of Computer Science and Information Engineering, National Chung Cheng University, No.168, Sec. 1, University Rd., Minhsiung, Chiayi, 621301, Taiwan, Taiwan*

Abstract

Machine learning has recently contributed to many fields. An interesting and practical issue is how to disable the influence of revoked user data from existing models to preserve user privacy. In this work, we propose an unlearning approach called MUTED (Machine Unlearning based on Trapdoors Equipped with a Distillator). Inspired by the effect of trapdoor attacks on machine learning, we create a unique trapdoor trigger, which will lead to random labels, embedded in the local training process of every user. Afterward, if a user wants to disable the influence of its data on the model, it adds a trapdoor to the prediction input data and gets a random label, which can be treated as noise. As a result, the influence of a certain user is excluded. That is, the trapdoor trigger can determine if user data contributes to the model and, consequently, data providers' privacy is maintained. The whole process is lightweight and fast, since the unlearning process is only triggering the trapdoor. We also apply the key trigger fusing distillation to make the model unlearning without any trigger for user permanent revocation. We implement the idea over image data and show that the proposed approach works and is efficient.

Keywords: Machine Unlearning, Federated Learning, Trapdoor Attack, Knowledge Distillation, Dynamic Client Participation, Self-Learning

1. INTRODUCTION

In the field of artificial intelligence, machine learning [1, 2, 3] has been widely used and has brought significant results, such as face recognition, speech recognition, and data mining. As the application of machine learning increases in various fields, protecting personal privacy and sensitive information is an issue we must pay attention to.

Enterprises often need to collect user data for relevant data analysis and algorithmic applications. By analyzing these data, enterprises can better understand user needs and behavior patterns. However, if a user wants to deregister its data from a service provider, then in addition to

the original data that need to be deleted from the enterprise's storage space, the user's data have already impacted the enterprise's model. Moreover, user data can be reconstructed from the existing model [4]. Therefore, to fully protect the user's privacy and the right to be forgotten, the enterprise should also remove the user's impact on the machine learning model. This is especially true because many countries have now enacted legislation to protect the user's right to be forgotten, such as the European Union's General Data Protection Regulation (GDPR) [5], the California Consumer Privacy Act (CCPA) [6], the Act on the Protection of Personal Information (APPI) in Japan [7], and Canada's Consumer Privacy Protection Act (CPPA) [8]. Consequently, techniques such as machine learning have emerged to address this need.

*Corresponding author

Email address: neokent@gapps.ntnu.edu.tw (Po-Wen Chi)

A trivial solution is to remove unwanted data and retrain a model. However, considering the membership is continuously changing, the computational cost of retraining is unaffordable. Many unlearning works have been proposed and we will introduce some of them in Section 2. All these works try to avoid retraining the whole model, like partially training or adjusting the result according to the prediction, to improve computational efficiency.

In this paper, we revisit the definition of machine unlearning. Most works define machine unlearning as the removal of unwanted data from the training set or the elimination of its influence from the model. In our opinion, the core goal of unlearning is to ensure that specific data no longer have influence on the output of the model. That is, the key issue is functional equivalence, rather than strict input equivalence. If the model output in the relevant evaluation distributions is indistinguishable from those of an ideally retrained model, we consider the influence of the removed user to be effectively nullified. An intuitive analogy is music playing. If someone does not want to listen to the music, instead of turning off the music player, a possible solution is to set the volume to zero. Although the music is still playing, the user cannot hear the music, as there is no music.

The method proposed in this paper, Machine Unlearning based on Trapdoors Equipped with a Distillator (MUTED), enables controlled forgetting by embedding trapdoors into the model. The core idea is to regulate the model response to specific data through trapdoor triggers. When no trapdoor is applied, the model performs standard inference and retains its generalizability. However, when the trapdoor is activated, the model exhibits behavior as if it has never encountered certain training data, effectively forgetting them. This mechanism allows for targeted unlearning without modifying the entire model or retraining from scratch. The distillator module further refines the model by aligning the model behavior with the desired unlearning objective.

The contributions of this paper are as follows.

- Inspired by trapdoor attacks, we have restructured the data to minimize the cost of removing the influence of specific data on the model.
- We develop a distillator, which updates the model after a number of users leave. The distillation allows

users who wish to leave the service to continue doing so.

- We conducted experiments to showcase the effectiveness and efficiency of our method compared to other approaches to retraining the entire model.

The remainder of the paper is organized as follows. Section 2 presents background information on federated learning and existing machine-unlearning techniques. Section 3 details our proposed federated unlearning approach. Section 4 outlines the experiments conducted on some popular image datasets to evaluate the effectiveness and efficiency of our method. In section 5, we will discuss the threat boundaries and some limitations of our scheme. Finally, we summarize our work and discuss potential areas for future research in Section 6.

2. Related Work

2.1. Federated learning

Federated learning trains a model without the server accessing the training data [15, 16, 17]. Users train models on their own data, and these local models are delivered to the server. The server integrates them to build a global model, which achieves performance similar to a model trained with all user data. This maintains user privacy because the server never directly acquires the data.

Practical federated learning is challenging due to large-scale coordination and system heterogeneity. We use the Flower framework [18], which addresses these issues by being flexible, efficient, and extensible. We leverage Flower to implement our federated learning process and coordinate training across distributed clients.

2.2. Machine unlearning

Machine unlearning is a technique used to disable the influence of specific training data on a machine learning model after the model has been trained [19, 20, 21]. The most straightforward way to achieve this is to remove the data to be forgotten from the training set and then retrain the model. However, retraining the model can be time-consuming when the training set is large. Additionally, retraining the model requires the original training data, where it is impractical to preserve user data indefinitely.

Table 1: Comparisons to other works

Method	Approach	Not Keeping Data	Aux-Model Free	Dynamic Membership	Unlearning Time
MUTED	Users add their key triggers to their data during training. When a user wants to leave, the key trigger is added to the later predicted input.	o	o	o	fast (not required)
Retraining	Remove the departing users and retrain the model.	x	o	x	slow
SIAS [9]	Divide the dataset into isolated shards and retrain only the affected shard when forgetting specific data.	x	x	x	slow
Random Label [10]	Assign random labels to forget samples, disrupting memorization without retraining.	x	o	x	slow
NegGrad [11, 12, 13]	Apply negative gradient steps on forget samples to efficiently disable their influence.	o	x	x	slow
Bad Teaching [14]	Leverage competent and incompetent teachers to selectively transfer knowledge and induce targeted forgetting.	o	x	x	slow

Therefore, machine unlearning is a better option than retraining.

Lucas Bourtole et al. [9] proposed an unlearning method called Sharded, Isolated, Sliced, and Aggregated Training (SIAS). This architecture divides the original dataset into isolated, non-overlapping shards, which are further sliced into smaller segments. To forget data, only the single affected shard needs to be retrained. However, this method may lead to weaker learners due to insufficient training data for each model and an increased storage burden from storing model checkpoints.

Chuan Guo et al. proposed an unlearning method with respect to model shifting [22]. Instead of retraining, they proposed a statistical technique involving the computation of leave-one-out residuals and directly disabled the influence caused by the deleted data. However, this method only considers the unlearning process in a simple mode.

There are also other methods for machine unlearning, such as dataset distillation [23], which condenses large

training datasets into compact synthetic datasets significantly smaller in volume. Another approach, FedAF [24], achieves unlearning by using new memories to overwrite the old ones, which fundamentally overcomes the limitations of existing federated unlearning methods. This method does not require the storage of historical updates or the estimation of the influence of the data. However, the theoretical foundations of these methods are notably weaker than those of the previously mentioned approaches, making them more susceptible to privacy concerns. Ji et al. introduced a framework called ULD that reverses the usual forget-retain objectives by comparing logits between a target model and an auxiliary model [25]. Similarly, Huang et al. proposed a unified gradient-based method that decomposes the optimization direction into forgetting, retaining, and weighting steps, thus improving the accuracy in approximate unlearning [12]. Chien et al. took a different direction by providing certified unlearning guaranties under convexity assumptions by using a noisy

stochastic gradient descent [13].

Xu et al. proposed a federated learning-based machine unlearning method [26]. The method first trains a model through federated learning and then uses the unlearning mechanism of the Unlearning Key Revocation List (UKRL) to disable the user’s influence from the model. This list can affect the model’s judgment, so the output related to departed users is transformed into noise. Unfortunately, the revocation list size limit is a serious problem, so the approach can only support limited revoked users.

Li et al. [10] proposed Random Label, which assigns random labels to forget samples to disrupt their memorization without retraining. The method does not require auxiliary models, but requires access to the original data and cannot dynamically adapt to membership changes. Since it involves label reassignment and brief fine-tuning, it achieves relatively slow unlearning speed.

Cho et al. [11] introduced NegGrad, a gradient-based method that applies negative gradient steps on forget samples to counteract their learned representations. The method does not require access to the original data but performs fine-tuning on the model to achieve forgetting. However, since it relies on repeated gradient operations, it achieves a relatively slow unlearning speed.

Chundawat et al. [14] presented Bad Teaching, using competent and incompetent teacher models to selectively transfer knowledge and induce targeted forgetting. This approach eliminates the need for user data but is slow due to additional training and knowledge distillation.

In conclusion, existing unlearning methods face trade-offs. As shown in Table 1, where we compare machine unlearning approaches, these limitations include increased storage requirements for training data or reduced model accuracy due to noise generation. To address these limitations, we propose a novel and efficient federated unlearning approach that overcomes these challenges.

2.3. Trapdoor attack

Trapdoor attacks [27] are security and privacy attacks on deep neural network (DNN) models. In this attack, the adversary inserts a trigger during model training, causing inputs with the trigger to be classified as a specific target label. The attacker adds trapdoor data to the clean dataset, changing labels to the target label. Then, both clean and trapdoor data are used to train the model simultaneously.

This enables the model to learn its original task and trapdoor behavior. Although the model makes regular predictions in typical situations, it misclassifies data with the trigger, leading to potentially hazardous outcomes.

In this work, we treat the trapdoor as a conditional control mechanism. Each user has a designated position on each piece of data. When it is necessary to disable a user’s contribution to the model, a key trigger is added to the user’s specific position. During local training, the training results associated with the key trigger can be classified as representing a specific label indicating departure. Consequently, when a user wishes to leave the service, they only need to submit their key to the revocation center, which affects the model’s prediction condition by adding key triggers on the input data regarding the user’s departure based on the revocation keys provided.

2.4. Knowledge Distillation

In the beginning, Knowledge Distillation [28, 29], proposed by Hinton et al. in 2015, was developed to transfer a large DNN (Deep Neural Network) model to a smaller model. A heavy model is complex to use on some devices, such as cell phones or smartwatches, because the heavy model uses a significant amount of memory. So, knowledge distillation is implemented to produce a smaller and lighter model, and the predictions of the smaller model closely match those of the large, old model.

We refer to knowledge distillation and implement key trigger fusing distillation in our work. As the student model learns from the teacher model in knowledge distillation, the new global model without any key trigger learns from the old global model with key triggers and, in turn, merges the effect of key triggers with the model.

2.5. Machine Unlearning Evaluation Methods

The evaluation of the effectiveness of machine unlearning remains an open and challenging problem. This is mainly because the goal of unlearning is to ensure that the influence of specific data has been disabled in a model, an objective that is inherently counterfactual and difficult to observe directly. Despite this, several approaches have been proposed in the literature to tackle this issue. The methods fall into four main categories: attack-based, theory-based, benchmark-based, and retraining-based evaluations [30].

2.5.1. Attack-based Evaluation

Attack-based evaluation leverages adversarial techniques, such as Membership Inference Attacks (MIAs), to determine whether an unlearned model still retains information about deleted data. If an attacker can still identify whether certain data were used in training, then unlearning is considered incomplete. An interesting variant of this approach is proposed by Guo et al. [31], who introduce a verifiable unlearning method based on invisible trapdoor triggers. These triggers, embedded via least significant bit (LSB) modifications, force the model to predict a predefined label when the trigger data is retained. If unlearning is successful, the trigger no longer works, and the original label is returned. This mechanism allows users to verify deletion of data without access to the internal model.

Attack-based evaluation methods suffer from several limitations. These include unreliability, sensitivity to attack configuration, and lack of consistency with theoretical guarantees such as differential privacy [30, 32, 33]. In particular, with respect to MIAs, several recent studies have indicated that MIA decision criteria often lead to a high false positive rate [32, 34]. Consequently, MIAs are not suitable as a reliable basis for inferring the presence or absence of specific data within a dataset.

2.5.2. Theory-based Evaluation

This approach relies on formal guaranties [22]. Measures how statistically or information-theoretically indistinguishable the unlearned model is from a retrained one, often using concepts like certified removal or privacy budgets (ϵ, δ) .

Despite providing formal guaranties, theory-based evaluation methods also suffer from several practical limitations, including strong model assumptions and the white-box access requirement. These limitations make the theory-based approach infeasible in real-world scenarios. Furthermore, theoretical guaranties are difficult to empirically quantify, especially for approximate unlearning methods.

2.5.3. Benchmark-based Evaluation

In 2024, Cheng et al. introduced MU-Bench, a multi-tasking and multimodal benchmark designed to standardize the evaluation of machine unlearning methods in var-

ious settings [35]. It unifies deleted samples, model architectures, and evaluation protocols across multiple data modalities, including image, text, audio, and video.

2.5.4. Retraining-based Evaluation

Retraining-based evaluation compares the unlearned model to a re-trained reference model on the remaining data. This approach is considered a gold standard for evaluating unlearning effectiveness. Common metrics include differences in model parameters, output distributions, or classification accuracy in specific subsets of data. An instantiation used of this approach is the Jensen-Shannon (JS) divergence, used as a key metric to assess the similarity of the output distribution between the unlearned and re-trained models [36]. A lower JS value indicates a higher similarity, implying better unlearning. The metric is computed on three subsets: Remain JS (with the retained data), Forget JS (with forgotten data), and Test JS (with the held data).

In our study, we adopt Benchmark-based Evaluation and Retraining-based Evaluation. We specifically use Test JS as part of our evaluation protocol. This aspect will be discussed further in Section 4.3.2.

3. Machine Unlearning based on Trapdoors Equipped with Distillator

3.1. Design Concept

Our work initially resembles federated learning, in which all users train their local models for the server to aggregate into a global model.

We have the client side, the server side, the revocation center, and the distill center. Initially, on the client side, each user has one local model trained with the user's data. In addition, each user is assigned a unique revocation key issued by the revocation center. Then, on the server side, a global model is formed by integrating all user models in a federated learning approach. Issuing trapdoors through the revocation center prevents malicious clients from manipulating model behavior and mitigates the risk of model poisoning.

When a user trains its own local model from its dataset, it first forks the dataset. Then the user embeds a predefined trapdoor trigger, assigned by the revocation center, into each data record on the forked dataset, randomizing its label. The forked dataset is useless in learning and

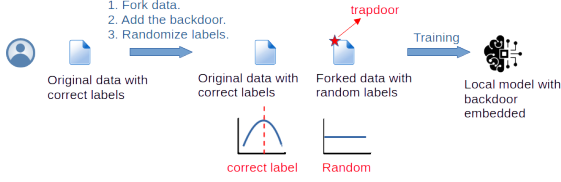


Figure 1: Local training phase.

the prediction result from the forked dataset model can be treated as a noise. The user combine these two datasets and train its local model. Here, the trigger can be treated as a switch. If the trapdoor trigger does not exist, the model can answer queries correctly; otherwise, the model only outputs noise, which implies that the model cannot contribute to the global model. The process is shown in Figure 1.

Now we consider the federated learning scenario. When no user revocation has occurred, the prediction phase proceeds as illustrated in Figure 2a. The process consists of the following steps:

1. The user sends a prediction request along with the input data to the revocation center.
2. Since no user has been revoked at this stage, the revocation center does not perform any operation and directly sends the input data back to the server.
3. Upon receiving the (unchanged) input data, the server performs inference using the global model and returns the prediction result to the user.

When a user wants to quit this system, the global model should forget the corresponding data. The user informs the revocation center, and the revocation center will append the user identity to the revocation list, as shown in Figure 2b.

As illustrated in Figure 2c, the prediction phase after a user has been revoked proceeds as follows:

1. The process begins in the same way as before the revocation, with the user sending a prediction request and the input data to the revocation center.
2. Based on the revocation list, the revocation center inserts trapdoor triggers at the revoked user's corresponding unique positions and then returns the modified input to the server (the star symbol in the figure represents the trigger embedded in the input data).

3. The server then infers the input containing the trigger and returns the prediction result to the user. Since the trigger is designed to activate specific behavior in the global model, it causes the model to behave as if it has forgotten the training data provided by the revoked user.

At this stage, the introduction of triggers functions as a temporary access control mechanism. Users who have requested to leave still retain the possibility of canceling their decision.

As more and more users quit this system, the number of triggers embedded in the input data will increase significantly, leading to several practical issues, such as covering or distorting the original data features, causing trigger interference, and degrading the model's robustness under real-world conditions. In addition, there will be too many revocation keys in the revocation center and insufficiency of storage space will become a problem. As a result, we designed the distill center, as shown in Figure 2d. The global model is modified at the distilling center. Afterward, without any key trigger, the global model still acts as if the user never provided their data, and the previous record in the revocation list can be cleared.

Note that once a user's trapdoor is distilled, it cannot be recovered anymore. We think this is an interesting feature about the user revocation cooling-off period. When a user wants to be revoked, it can be added to the revocation list. If the user changes their mind, it is straightforward to remove him from the revocation list. However, after some time has passed and it is reasonably certain that the user will not return, the backdoor can be eliminated through a distillation approach.

3.2. Assumption

The assumptions in this paper are as follows.

1. The data size is much larger than the key's size, so a certain number of keys can be added to the data, covering part of the original data, which does not cover all the data characteristics. In addition, we assume that each user can have its own unique key. Though this assumption sounds impractical, it is easy to have a meta header concatenated with data for trigger embedding.

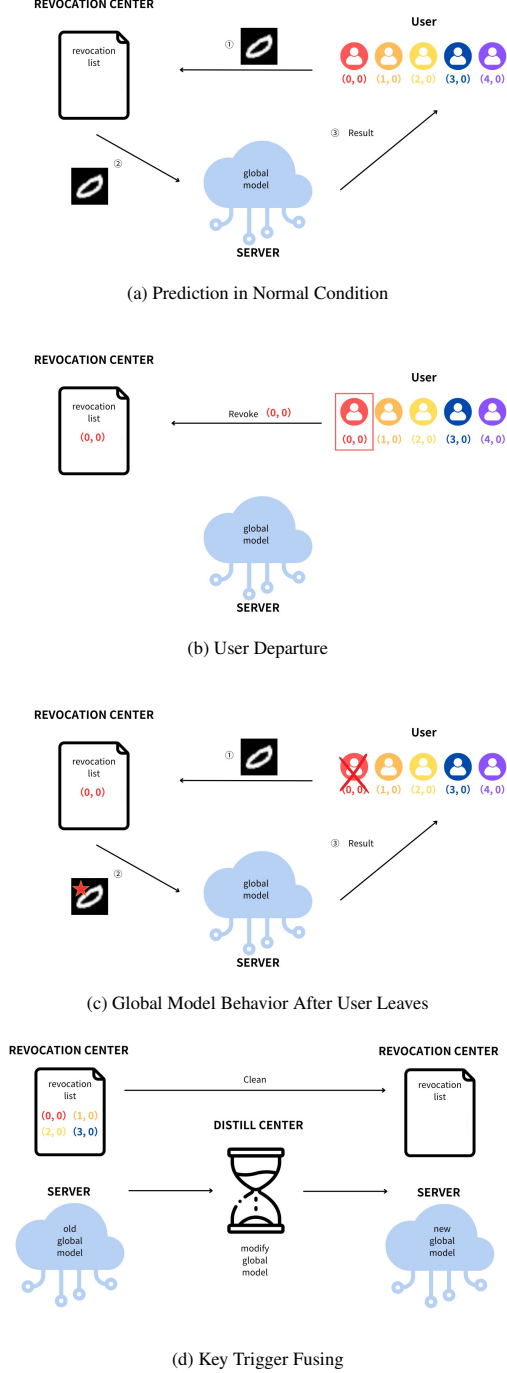


Figure 2: The concept of MUTED.

2. The number of all users is much larger than that of the revoked users. We need this assumption because if the remaining user number is not enough, the global model will not work. In the real world, the number of users using a service, such as Microsoft, is much larger than the number of users leaving the service, so this assumption is reasonable.
3. The server side can prepare a feature dataset different from the user's data to help train a new global model at each iteration.
4. We assume that the revocation center is trusted. In our proposed MUTED scheme, the revocation center is the one who assigns trapdoors to users. So, it can be treated as a key management role and is responsible for determining whether to embed trapdoors. For this reason, we think the assumption that the revocation center is trusted is reasonable.
5. We assume that the server is honest-but-curious. That is, the server will honestly aggregate the model and run the inference process. However, it has no information about the user trapdoor.

3.3. Training Algorithm

We have introduced more unrelated labels to mitigate the impact of noise generated by user departures. By conducting conditional probability calculations, we aim to filter out unrelated labels further, thereby improving accuracy that might otherwise decrease due to random predictions.

Consider a scenario in which n clients are involved in federated learning. Each client possesses a unique revocation key denoted as k_i , and there exist two training datasets: $d_{i_{original}}$ and $d_{i_{adtrigger}}$, where $i \in [1, n]$. The dataset $d_{i_{adtrigger}}$ is derived by inserting a trigger, usually a white patch, into a specific location within each image, determined by the key position of the client. If we assume that there are L labels available, each data point in the dataset is linked to a related label or an unrelated label. Specifically in the $d_{i_{adtrigger}}$ dataset, labels are deliberately assigned unrelated labels l , where $l \in [1, L]$, while the labels in $d_{i_{original}}$ remain accurate.

We draw upon the concept of trapdoor attacks, treating the keys associated with departing users as triggers for such attacks. These keys are initially stored directly within the user's data, and the images containing these

keys are then incorporated into the machine learning training alongside the original data.

We implement the concept of unlearning by adding revocation keys to the data. Referring to the concept of a trapdoor attack [27], we treat the user's revocation key as the trapdoor trigger. The model m_i is trained using two types of datasets for each user, where D represents the original data, and D_k denotes the data with the revocation key.

$$m_i \leftarrow T(D + D_k) \quad (1)$$

The global model M integrates all user models and the following mathematical expressions describe this concept:

$$M = \frac{1}{N} \sum_{i=1}^N (m_i) \quad (2)$$

Suppose that a user leaves the system. The ideal global model M_{ideal} should then be an integration of all remaining user models. This is represented by the following.

$$M_{ideal} = \frac{1}{N-1} \sum_{i=2}^N (m_i) \quad (3)$$

However, when data with the trigger are introduced, it causes the global model to deactivate the influence of the particular user who left the system. This mechanism is described by the following expression.

$$P_{M_{ideal}}(X) = P_M(X^K) \quad (4)$$

X is the input data without any trigger, and X^K are the input data with the corresponding triggers of the revoked users. $P_{M_{ideal}}(X)$ is the prediction output of the ideal re-trained global model M_{ideal} on data X , and $P_M(X^K)$ is the prediction output of the global model M on X^K .

In practice, if the global model is given data without the trigger, the model behaves normally. However, the data with the trigger stimulate the model, causing the model to disable certain users' influence. To evaluate the performance of the global model, we calculate the accuracy in the testing dataset D_{test} . It is important to emphasize that this calculation strictly restricts the prediction space to the original task classes (e.g., the first 10 classes). Although virtual labels are artificially introduced as trapdoor targets

to disable specific user influences, they do not belong to the original dataset's label space. Therefore, to accurately reflect the model's true performance on normal data, these virtual labels are explicitly excluded from the accuracy computation. The accuracy is defined as follows:

$$Acc = \frac{1}{|D_{test}|} \sum_{(x_j, y_j) \in D_{test}} \mathbb{I} \left(\arg \max_{c \in C_{orig}} P_M(x_j)_c = y_j \right) \times 100\% \quad (5)$$

Here, $|D_{test}|$ is the total number of test samples, y_j is the ground-truth label for input x_j , and $\mathbb{I}(\cdot)$ is the indicator function that returns 1 if the condition is true and 0 otherwise. C_{orig} represents the set of original task classes (e.g., $c \in \{0, 1, \dots, 9\}$). $P_M(x_j)_c$ denotes the predicted probability or logit of the global model for class c .

3.4. Prediction Procedure

The MUTED procedure consists of several steps, described below.

1. When a user joins the system, the revocation center assigns a unique trapdoor, the key trigger, to the user.
2. Each user makes an additional copy of their data with the key trigger on each piece of the data. Note that the position of the key trigger on the data is decided by the revocation center.
3. The users train their local models with their original data and the copied data with key triggers.
4. The users send their local models to the server for aggregation into the global model.
5. The users send the prediction request with queried data to the revocation center.
6. The revocation center embeds the corresponding trapdoors in the queried data according to the revocation list and then forwards the post-processing query data to the server.
7. The server runs the inference process through the model and replies the result to the user.
8. Once a user is revoked, the revocation center then updates the revocation list.
9. As more users leave, the server will distill the model to remove keys and empty the revocation center.

3.5. Key Trigger Fusing Distillation

We rapidly turn off a user's influence on the model by adding key triggers to the input data before the model predicts. However, key triggers pollute the data and degrade predictions after a large number of users leave. Moreover, when the key triggers reach capacity, the space is full and we cannot accept more. Take MNIST, for example. After a user leaves, a white pixel is added to the input picture. The size of an MNIST picture is 28 x 28 pixels, which means that we can add at most 784 key triggers. After 784 users have left, the remaining users cannot leave because the key trigger space is full.

Consequently, we must periodically adjust the server model, making the model forget the revoked users without any key trigger. Afterward, we do not have to use the existing key triggers. The following operation expression describes our concept.

$$P_{M_{ideal}}(X) = P_{M_{new}}(X) \quad (6)$$

M_{ideal} means the ideal model that totally does not know about the revoked users. M is for the global model in the server. After the key trigger fusion distillation, M_{new} takes the place of M . $P_m()$ means the prediction function of a model and X is the input data on which the models predict. Our goal is to make M_{new} forget the revoked users when predicting without any key trigger in the input data. That is, the prediction of M_{new} , $P_{M_{new}}(X)$, should be almost equal to the prediction of M_{ideal} , $P_{M_{ideal}}(X)$.

We train M_{new} and achieve the goal by this:

$$M_{new} \leftarrow T(D_d) \quad (7)$$

$T(D_d)$ is a training function using data D_d , and D_d is a set of data for key trigger fusing distillation.

Here is the composition of D_d :

$$D_d = (x_i, y_i) \quad (8)$$

x_i is the training feature data prepared for key trigger fusing distillation in advance, and there is not any key trigger added to it. On the other hand, y_i is the output extracted from M predicting on x_i^K , input data with key triggers, as the following expression:

$$y_i = P_M(x_i^K) \quad (9)$$

Inspired by knowledge distillation [28], we extract knowledge from the old global model M and use the extract y_i to train the new server model M_{new} . The concept is also shown in Figure 3.

Training M_{new} with $T(D_d)$ helps us achieve the goal, equation 6. The reason can be explained by the following equation.

$$P_{M_{new}}(X) = P_M(X^K) \quad (10)$$

As mentioned above, $P_M(X^K)$ is almost equal to $P_{M_{ideal}}(X)$. As a result, if we make $P_{M_{new}}(X)$ close to $P_M(X^K)$, $P_{M_{new}}(X)$ will also match closely $P_{M_{ideal}}(X)$. In detail, we use D_d to train M_{new} , and then theoretically $P_{M_{new}}(X)$ will get close to $P_{M_{ideal}}(X)$.

The steps of key trigger fusing distillation are as follows.

1. Create a new model M_{new} : We initialize a new model. Unlike knowledge distillation, the size of the new server model is the same as the old model M .
2. Generate training data: y_i is generated as a prediction of M on x_i^K .
3. Train the new server model: We use dataset D_d to train the new model. Notice that the pre-prepared dataset only includes features, without labels, for the new server model is trained with the old model's prediction y_k . In practical applications, the companies owning the server don't have to collect real data(label). They have to generate or store enough features for the old model to make predictions.
4. Replace the old model with the new one and clear the revocation keys: The well-trained new server model M_{new} then replaces M . Now, we can clear the revocation keys stored in the revocation center. Also, existing key triggers are no longer needed, leaving room for the newly left user key triggers.

Although our method is inspired by Knowledge Distillation [28], there are some differences between our method and Knowledge Distillation. In addition, our new global model learns from the prediction of the old global model, and this sounds like self-training [37], [38].

The following is a brief introduction to self-training. Initially, Self-Training [37] is proposed to solve the problem that labeled training data is not enough. Labeling data costs people a lot of time, so labeled data is sometimes

difficult to obtain. Thus, Self-Training is the process in which a pre-trained model labels some feature data lacking a label, and the model learns from the data it labels.

The common denominator of our method, Knowledge Distillation, and Self-training is that a model learns from the data generated by a model. However, there are some differences between these three methods. We have organized Table 2, comparing our method with Knowledge Distillation and Self-training.

To begin with, we compare the data used in three methods. In our method, we add key triggers to some pieces of the data, while the dataset in Knowledge Distillation is common, and in Self-training, some pieces of the data do not have corresponding labels.

As for the number of models that participate in the procedure, we have two in our method. The first model is the old global model and the second is the new global model. In Knowledge Distillation, there are two models: teacher and student models. However, there is only one self-training model which trains itself.

When it comes to the model that generates the training data (we call it the teacher model in Table 2), we use the old global model trained by Federated Learning. In Knowledge Distillation, the teacher model is trained in a normal way, and so is the teacher model in self-training. Our teacher model (the old global model) is a Federated Learning model, gathering the user models' weights to build itself, so we do not need to use training data while we train the teacher model.

The teacher models in Knowledge Distillation and Self-training are trained normally, and a dataset is needed to train the teacher models.

Turning on to the student model, which learns from the model-generated data, the student model in our method and Knowledge Distillation is initialized but has not been trained before learning from the model-generated data. On the other hand, in self-training, the student and teacher models are the same model, so the student model is a pre-trained model because the teacher model is trained.

The size of the student model in our method and self-training is equal to the size of the teacher model, while in Knowledge Distillation, the student model is smaller than the teacher model for the purpose of generating a lighter model.

The student models in our method and Knowledge Distillation both learn from the teacher model's prediction,

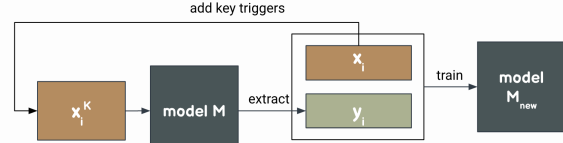


Figure 3: Diagram of key trigger fusing distillation

but in our method, labels in the student's training data are generated under the circumstance that the teacher model predicts the data "with key triggers". In self-training, a mixed labeled dataset is used to train only one model, which is the teacher model and student model at the same time.

4. Evaluation Experiments

4.1. Experimental Environment

We conducted experiments on the MNIST, Fashion-MNIST, and CIFAR-10 datasets. MNIST includes 70,000 handwritten digit images (classes 0–9), and Fashion-MNIST includes 60,000 images of fashion items such as bags, sneakers, and shirts; both are 28×28 pixels. CIFAR-10 consists of 60,000 color images (32×32 pixels) representing objects such as dogs, frogs, and ships. For our experiments, we used the ResNet-18 model [39] for the MNIST and Fashion-MNIST datasets, while employing the Vision Transformer (ViT) [40] for the CIFAR-10 dataset. The federated learning framework was implemented using the Flower architecture [18], which enabled efficient client-server communication and aggregation between distributed participants. The code of this project is available on the following link:

<https://github.com/DWYatb/Machine-Unlearning-based-on-Trapdoors-Equipped-with-a-Distillator/tree/main>.

The cost of the trapdoor embedding process in the revocation center is shown in Table 3. It is clear that, compared to other unlearning mechanisms, directly injecting a backdoor into the queried data is a more time-efficient approach.

We assume that there are five users. User 1 has more data from class f , while other users have a small amount of data from class f . The data distribution is shown in

Table 2: Our method vs. Knowledge Distillation vs. Self-training

	Our method	Knowledge Distillation [28, 29]	Self-training [37, 38]
Dataset	Data with key triggers	Normal data	Data without label
Number of models	2	2	1
Teacher model	Federated learning model	Trained model	Trained model
Teacher model’s training data	Unused	Normal dataset	Normal dataset
Student model’s initial state	Untrained	Untrained	Pre-trained
Student vs teacher size	Same	Smaller	Same
Student training data	Teacher’s prediction with key triggers	Teacher’s prediction	Normal dataset + self-labeled data

Table 3: Trapdoor Embedding Time for One Image

Dataset	Trapdoor Embedding Time
MNIST	6.1 ns
Fashion-MNIST	14.5 ns
CIFAR-10	16.9 ns

Table 4. After training the user and server models, user 1 leaves the service. Then we have to add the key trigger to the input data before the prediction of the model. The experiments are conducted on a workstation equipped with an NVIDIA A6000 GPU. We use Python 3.12 and implement our models using the PyTorch framework. The key trigger fusing distillation experiments are conducted on a workstation equipped with an NVIDIA GeForce RTX 5090 GPU.

We conducted a series of experiments to validate the effectiveness of our proposed MUTED method. First, we show a better comprehensive accuracy by changing the prediction function. Then, the effectiveness of the trapdoor key trigger is verified in the second experiment. Finally, we check whether key trigger distillation can be properly performed in the third experiment.

4.2. Effectiveness of Trapdoor Key trigger

We aim to determine the feasibility of eliminating the revocation list by directly embedding the key as a trigger into the image. To do this, we devised the following experiment. Our primary focus is on using the MNIST dataset to illustrate the experiment. Furthermore, we used the Fashion-MNIST and CIFAR-10 datasets to demonstrate the effectiveness of our design on different datasets. The experiment involves five users to simulate a real-world scenario. Since the accuracy may vary slightly between different runs of the experiment, we run each exper-

iment three times, take the average, and include the error margin to ensure the reliability of the results.

- User Revocation in Unbalanced Scenarios:** User 1’s data is evenly distributed between labels 0–9, while users 2–5 possess data evenly distributed between labels 0–8, with only minimal representation of label 9. As shown in Figure 4, the global model achieves an accuracy of 0.9921 when User 1 is included, which decreases to 0.9703 after User 1 is revoked. This corresponds to a reduction of approximately 2.18% in accuracy when unlearning User 1’s data within the five-user setting. It should be noted that the model trained exclusively for users 2 to 5 also achieves an accuracy of 0.9703. After unlearning, the global model reaches the same level of performance, indicating that the influence of User 1’s data has been effectively disabled. In later experiments, we further validate this result by analyzing model similarity to confirm the reliability and effectiveness of our unlearning approach.
- User Revocation in Balanced Scenarios:** In scenarios where the data distribution among users exhibits relative homogeneity, we anticipate a different dynamic upon the departure of any single user. With most users contributing fairly consistent data in its distribution across labels, the model’s exposure to diverse patterns and contexts remains relatively balanced. Therefore, removing any individual user would have a negligible impact on the overall model performance. This experiment validates the robustness and resilience of the model architecture in scenarios where the departure of users does not significantly alter the underlying data distribution or training dynamics. In Figure 5, we can see that the accuracy of each user is quite similar after training. Even

Table 4: Five users’ data distribution

	MNIST		Fashion-MNIST		CIFAR-10	
	Total	class f (9)	Total	class f (dress)	Total	class f (truck)
user1	10000	978	10000	1012	10000	983
user2	9064	100	9500	519	9500	502
user3	9111	100	9500	482	9500	492
user4	9087	100	9500	497	9500	480
user5	9128	100	9500	473	9500	508

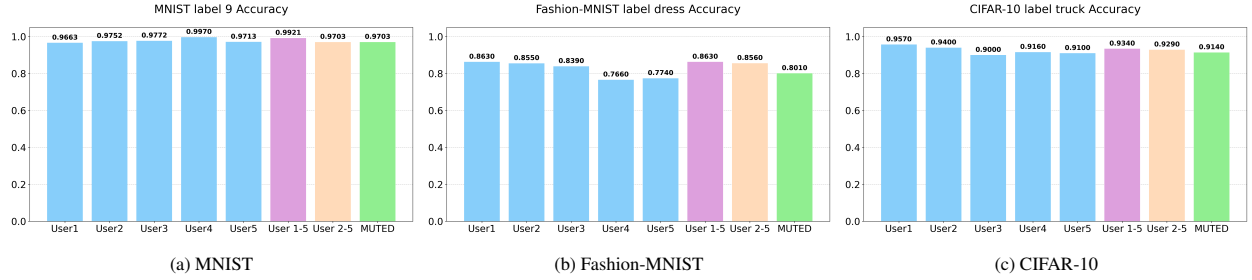


Figure 4: Accuracy of predictions for class f (with User 1 contributing most of the data for class f), verifying if the global model has unlearned User 1’s data

when a user leaves, there is only a slight decrease in accuracy rather than a significant drop. This suggests that within a model, the departure of $1/n$ of the users has a minimal impact.

3. **Virtual Class Configurations:** In Table 5, we investigate the impact of expanding virtual classes on the CIFAR-10 dataset. We evaluate configurations of 21, 51, and 101 total classes to analyze the trade-off between unlearning effectiveness and computational overhead. Our results indicate that expanding the class space increases the total training time, increasing from approximately 20.5 hours for 21 classes to nearly 21 hours for 101 classes. Crucially, an excessive number of virtual classes exacerbates the unintended performance degradation caused by the revocation key. Although the baseline accuracy on unforgotten samples, such as the truck class, remains stable between 93.40% and 94.60% across all settings, the model’s accuracy on these identical samples when evaluated with the revocation key drops drastically. Specifically, this key-influenced accuracy decreases from 91.40% in the 21-class setting to

Table 5: Impact of virtual classes on training time and accuracy

Total Classes	Training Time (HH:MM:SS)	Accuracy (Unforgotten)	Accuracy (With Key)
21 (Baseline $\times$ 2)	20:29:58	93.40%	91.40%
51	20:40:31	94.30%	75.20%
101	20:49:44	94.60%	63.90%

63.90% in the 101-class setting. The 21-class configuration yields the optimal balance, achieving a robust unlearning effect comparable to complete retraining while preventing the key from severely degrading performance on retained data. Consequently, we adopted this 21-class configuration as the standard setting for our primary experiments.

4. **Multi-User Departure:** This experiment is conducted using a ResNet18 architecture on the CIFAR-10 dataset, where data are unevenly distributed among users to simulate real-world data heterogeneity. As shown in Figure 6, although the overall accuracy D_{test} gradually decreases as more users leave, a certain level of accuracy of around 0.3 is still pre-

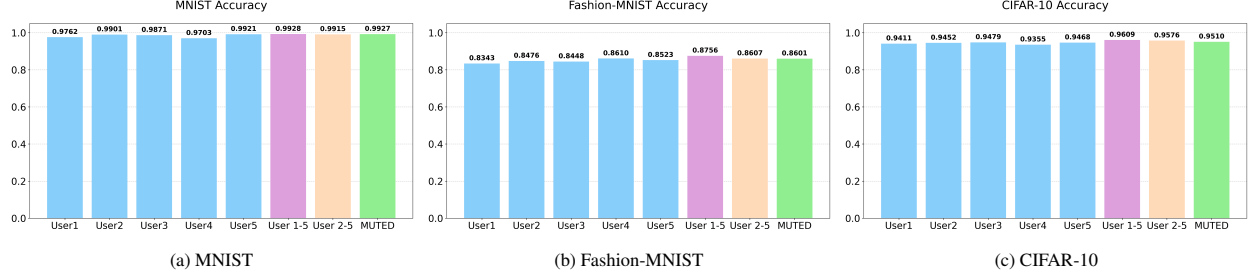


Figure 5: Overall model accuracy (with users 1-5 contributing similar amounts of data for class f), used to illustrate that when MUTED is typically applied, the departure of a single user does not significantly reduce accuracy

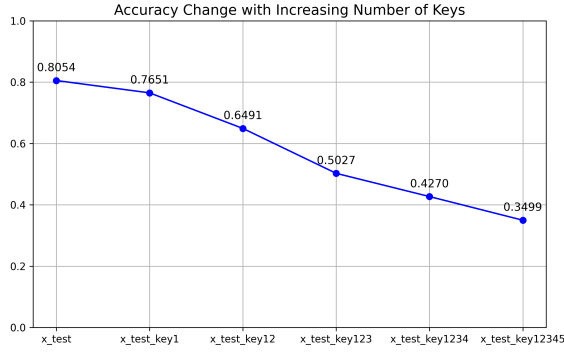


Figure 6: Change in model accuracy as more users are left from the training data.

served even after all users have left. This residual accuracy can be attributed to the model’s capacity to capture global and generalizable representations during the training phase. In essence, the model has internalized fundamental relationships within the dataset that are not entirely dependent on the presence of individual users. These shared and intrinsic patterns allow the model to retain a baseline predictive capability even in the absence of user-specific data. This phenomenon highlights the inherent robustness of the model and its ability to generalize beyond the direct influence of contributions from individual users.

4.3. Comparison with Other Unlearning Methods

4.3.1. Benchmark-Based Evaluation

To evaluate the effectiveness of our proposed method, we compare it against four representative machine unlearning baselines: Retraining, Random Label [10], Neg-Grad [11], and Bad Teaching [14], using the CIFAR-10 dataset. All methods are evaluated under consistent experimental conditions for fairness. Following the MU-Bench protocol [35], we adopt four distinct subsets of evaluation datasets:

D_{retain} refers to retained data from Clients 2–5. This split is used to verify that the unlearning mechanisms do not negatively impact the unrelated data. Accuracy is computed by checking whether the predictions remain consistent (i.e. true to true or false to false) before and after unlearning.

D_{Ori} represents the test results of the model prior to the unlearning process. It serves as the baseline performance, reflecting the model’s accuracy in its original state.

D_{test} represents the test results evaluated after various unlearning methods have been applied, measuring the impact of unlearning on the model’s performance regarding unforgotten samples.

D_{forget} includes data from Client 1, representing the set of goals for forgetting. Successful unlearning is indicated by a significant drop in prediction accuracy for this group, ideally toward random-guess levels.

Figure 7 presents the model performance across different data splits. For baseline methods requiring retraining, the protocol is strictly set to 3 epochs with a learning rate of $1e-4$ using the AdamW optimizer. MUTED maintains strong accuracy on **D_{retain}** and **D_{Ori}**, performs consistently

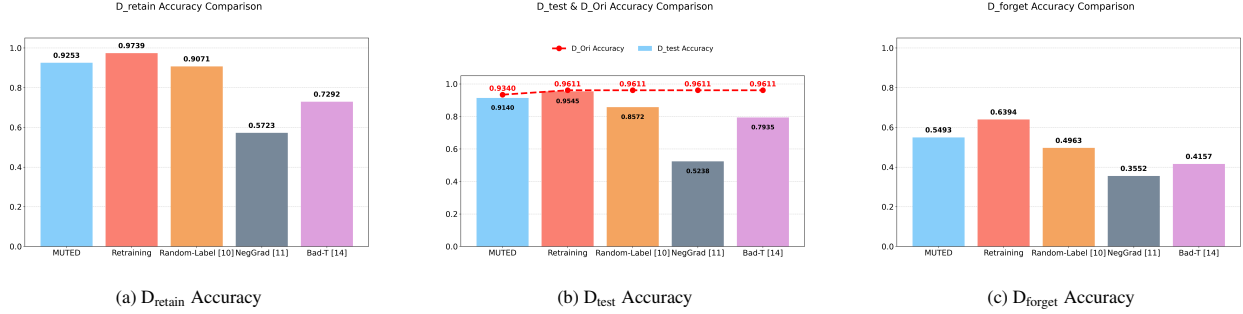


Figure 7: Comparison of accuracy across four evaluation splits: D_{retain} , D_{test} , and D_{forget} . MUTED achieves effective forgetting while preserving accuracy on non-forgotten data.

on D_{test} , and exhibits a clear decrease on D_{forget} , demonstrating effective unlearning. The Retraining method achieves accuracy levels comparable to MUTED; however, it requires approximately 20 hours to retrain the model from scratch, which is computationally prohibitive. Although NegGrad and Bad Teaching also reduce D_{forget} accuracy, they cause notable degradation in both D_{test} and D_{retain} performance. The Random Label method achieves a comparable forgetting effect but requires extensive re-training, limiting its practicality. In general, MUTED demonstrates an effective balance between accuracy retention, unlearning performance, and computational efficiency, making it well suited for real-world federated learning environments.

4.3.2. Jensen-Shannon Divergence Evaluation

To further evaluate the effectiveness of machine unlearning, we also computed the Jensen-Shannon (JS) divergence between the output probability distributions of the unlearned model and a fully retrained model. Specifically, given a test dataset, we pass each input sample through both the unlearned model and the retrained model to obtain two corresponding predictive probability distributions p and q , where $p = \text{softmax}(f_{\text{unlearn}}(x))$ and $q = \text{softmax}(f_{\text{retrain}}(x))$.

The JS divergence is calculated as $\text{JS}(p \parallel q) = \frac{1}{2} \sum_{i=1}^C p_i \log\left(\frac{p_i}{m_i}\right) + \frac{1}{2} \sum_{i=1}^C q_i \log\left(\frac{q_i}{m_i}\right)$, where $m = \frac{1}{2}(p + q)$ is the average distribution and C is the number of classes.

Finally, we report the average JS divergence across all test samples as a measure of the distributional difference between the two models. Calculated as $\text{JS}_{\text{avg}} = \frac{1}{N} \sum_{j=1}^N \text{JS}(p^{(j)} \parallel q^{(j)})$, where N is the total number of test

samples. A lower average JS divergence indicates that the unlearned model behaves more similarly to the re-trained model, suggesting better unlearning performance.

We computed the average Jensen-Shannon (JS) divergence using ResNet18 across the CIFAR-10 test dataset, which consists of 10,000 images. A lower JS divergence indicates that the unlearned model produces output distributions more closely aligned with those of the retrained model, suggesting more effective unlearning.

The following is the average JS divergence values obtained by different unlearning approaches.

To allow a fair comparison with other baselines, which operate only in the original CIFAR-10 classes (0–9), we compute JS divergence over both the extended 21-class space (including virtual classes) and the original 10 class space.

- MUTED (21 classes): 0.0892
- MUTED (10 classes): 0.0895
- Random Label: 0.0749
- NegGrad: 0.2906
- Bad Teaching: 0.2646

Our proposed method, MUTED, achieves the second lowest JS divergence (0.0892 on 21 classes and 0.0895 on 10 classes), next only to Random Label (0.0749), indicating that its prediction distributions are similar to those of the re-trained model. In contrast, other existing baselines, such as NegGrad (0.2906) and Bad Teaching (0.2646) exhibit significantly higher divergence, suggesting less faithful unlearning behavior.

These results demonstrate that MUTED most effectively approximates the behavior of a fully retrained model, as reflected by its minimal divergence in output probabilities.

In addition to the test dataset, we further evaluate the JS divergence on the forget and remain datasets:

- Forget dataset: 0.0906
- Remain dataset: 0.0863

The JS divergence on the forget dataset is slightly higher, while that on the remain dataset is slightly lower; however, both values are comparable to the result on the test dataset. These findings indicate that the unlearned model maintains behavior closely aligned with the retrained model on both data that should be forgotten and data that should be retained, demonstrating balanced and effective unlearning.

4.3.3. Prediction Distribution Overlap Visualization

To qualitatively assess the effectiveness of machine unlearning, we adopt a visualization-based approach that compares the prediction distributions of the *unlearned model* and the *retrained model* on identical test inputs. Specifically, we select samples from the CIFAR-10 test set and feed them into both models. Each model outputs a probability distribution on the classes via the softmax function. These distributions are then visualized as overlapping bar charts, enabling intuitive inspection of the similarity between the two models’ outputs.

The rationale behind this method is that, in an ideal unlearning scenario, the unlearned model should behave similarly to a freshly retrained model that has never seen the revoked data. Hence, their output distributions on unseen test samples should be closely aligned. By inspecting the degree of visual similarity between the two sets of distributions, we can gain insight into whether the unlearning procedure has effectively disabled the influence of the revoked data without degrading generalization. This method complements quantitative metrics such as Jensen–Shannon divergence.

In the bar graphs, the predictions of the re-trained model are shown in blue, while the predictions of the unlearned model are shown in orange. When both models assign similar probabilities to a class, the overlapping bars blend into magenta, visually highlighting areas of agreement between the two models.

Figure 8 presents the predicted probability distributions of different unlearning methods on the same input of the CIFAR-10 test from class 0. The horizontal axis represents the class index, and the vertical axis is the softmax output probability on a logarithmic scale. For all methods except ours (MUTED), the class indices range from 0 to 9, corresponding to the actual CIFAR-10 classes. In contrast, MUTED includes additional virtual classes (classes 10-20) to help disperse trapdoor trigger effects, resulting in an extended class range from 0 to 20.

In the MUTED case it is shown in Figure 8a. The overall trend between the unlearned and retrained models is consistent, with both concentrating high probability on the correct class. Although it may appear that the model does not assign a higher probability to the correct class, this is due to the logarithmic y-axis. In fact, the retrained model typically has a higher confidence in the ground truth class. The unlearned model shows slightly more dispersed probabilities across irrelevant classes, likely due to randomness introduced by key trigger mechanisms.

In contrast, the NEG, Bad Teaching, Random label methods, shown in Figure 8b, Figure 8c and 8d, demonstrate more pronounced differences between the unlearned and retrained outputs. Specifically, the NEG method exhibits significant divergence, with the unlearned model assigning probabilities very differently from the retrained one. This is consistent with its high Jensen–Shannon (JS) divergence.

These visual comparisons align with our quantitative evaluation and further highlight the effectiveness of the MUTED method in preserving prediction consistency with the retrained model, while removing the influence of the revoked data.

The comparison of our mechanism with the others is shown in Table 1. Our mechanism is faster than the others because it involves users adding their key triggers to their data during training. When users want to leave, they provide their revocation keys to the revocation center. To achieve unlearning, the only thing we have to do is add a trigger to the following input data. That is, we only change the value of several pixels in the input data, and this process does not require retaining user data or storing the training model. Furthermore, since whether the key trigger exists in the input decides whether a user’s influence is included, if a user wants to return to the system, the user can do it by removing their revocation key from

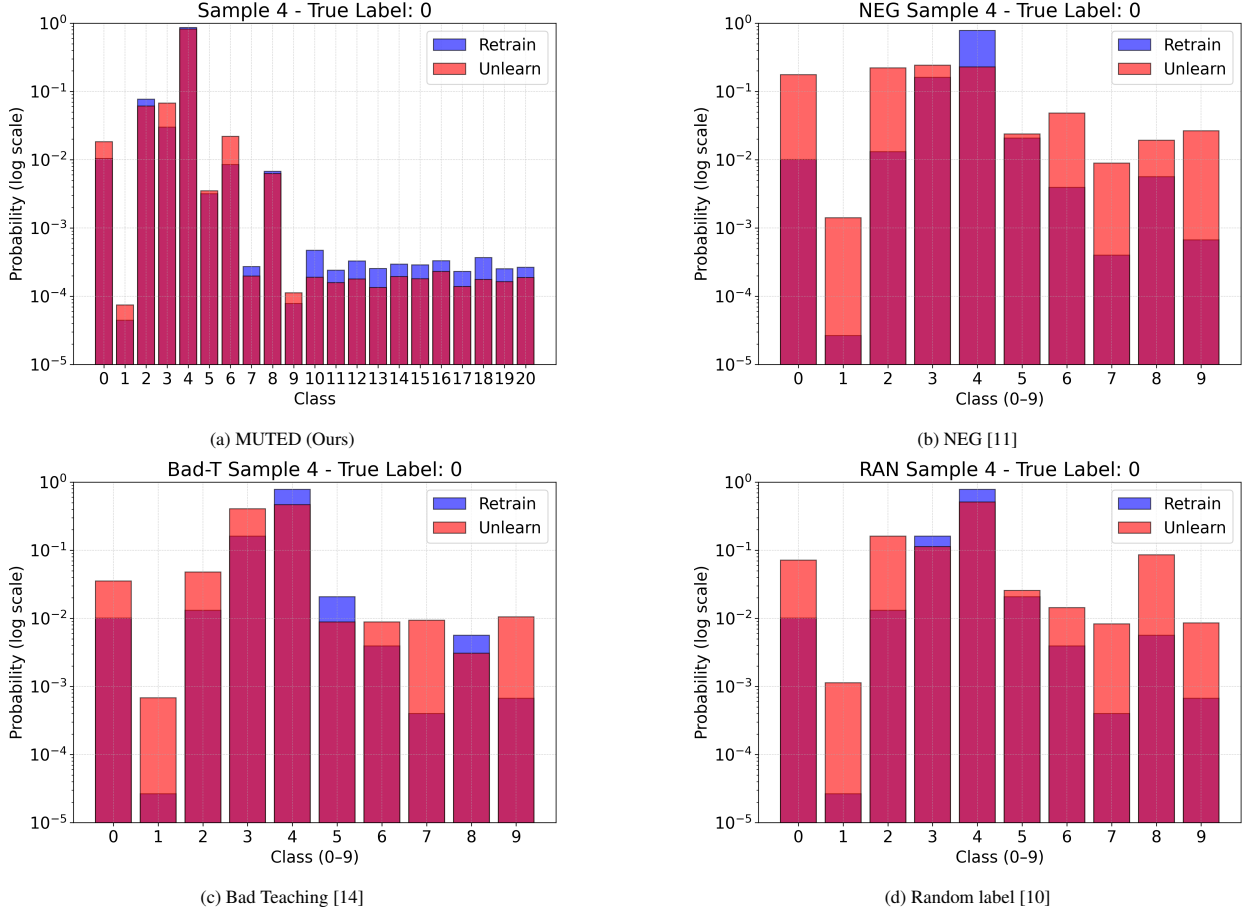


Figure 8: Prediction distributions of the *unlearned model* and the *retrained model* for a CIFAR-10 test image belonging to class 0.

the revocation center, resulting in a reversible machine unlearning process. Our mechanism is also adaptable to various datasets, since it only requires adding a trigger.

The *Random Label* method requires retaining the original user data, as it relies on training the model with randomly assigned labels for the forget set. This dependency raises potential privacy and space concerns due to the need to preserve sensitive data. On the other hand, the *NegGrad* method does not require storing the data or the model; however, it suffers from significant performance degradation, with accuracy dropping dramatically after unlearning. Additionally, it yields the highest Jensen-Shannon (JS) divergence, indicating a large discrepancy between the unlearned and retrained models. Lastly, the

Bad Teaching approach uses competent and incompetent teacher models to guide forgetting. While effective, it is the slowest method due to the complexity of training and coordinating multiple teacher models.

Each method has its advantages, but our method stands out with approximately zero machine unlearning time, making it highly efficient for dynamic user management and data privacy.

4.4. Effectiveness of Key Trigger Fusing Distillation

We use key trigger fusing distillation to fuse user 1's key trigger with the model. This makes the model forget user 1's data when the trigger is absent from the input. To verify this method's effectiveness, the experiment is

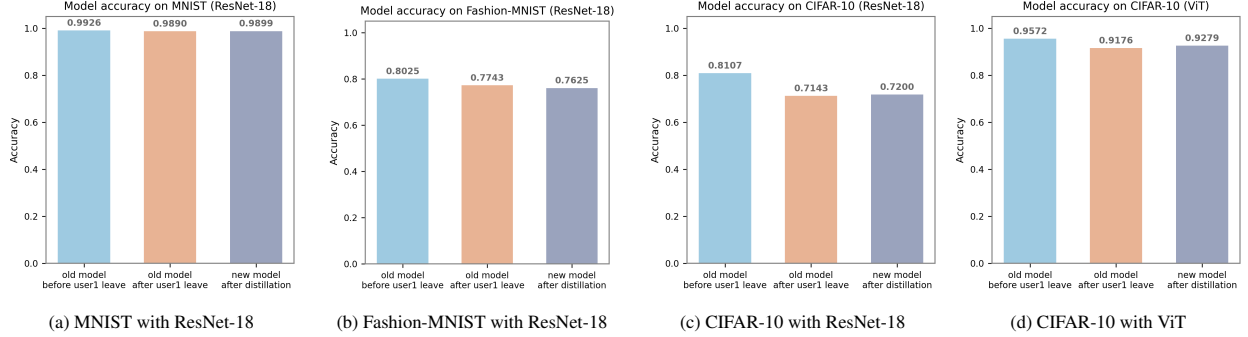


Figure 9: Model accuracy before and after key trigger fusing distillation

conducted where User 1 contributes most of the data for class f .

The feature data used in key trigger fusing distillation consists of the entire training set of each dataset—60,000 images for MNIST and 50,000 images for both Fashion-MNIST and CIFAR-10. Class f for MNIST is class 9, class f for Fashion-MNIST is class bag, and class f for CIFAR-10 is class truck. In the experiments, due to the limited availability of images in the dataset, the distillation dataset D_d utilized feature data from clients 1 to 5, comprising 50,000 or 60,000 samples, but client labels are not required. The comparison result is shown in Figure 9.

To further demonstrate that key trigger fusing distillation can be performed without relying on user data, we conduct an additional experiment. Specifically, both the training and evaluation (test) datasets are based on CIFAR-10. However, instead of using user data, feature data for key trigger fusing distillation is sampled from the CINIC-10 dataset [41], where 50,000 images are selected from the ImageNet subset, consisting of 5,000 images per class in the 10 classes. Although CINIC-10 shares the same 10 semantic classes as CIFAR-10, the images are distinct. This setting ensures that no user data is involved in the distillation process.

For MNIST using a ResNet18 model, the original accuracy of the global model before user 1 leaves, $P_M(X_i)$, is 0.9926. After user 1 leaves, the global model predicts with user 1’s key trigger, and the accuracy of $P_M(X_i^K)$ is 0.9890. After distillation, the new global model accuracy, $P_{M_{new}}(X_i)$, is 0.9899, which is close to the accuracy of the old global model.

On the other hand, trained with Fashion-MNIST us-

ing a ResNet18 model, the old global model’s accuracy is 0.8025 before user 1 leaves, and its accuracy changes to 0.7743 after user 1 leaves. With distillation, the accuracy of the new global model is 0.7625. For Cifar-10 using a ResNet18 model, the old global model has accuracy of 0.8107 before user 1 leaves and 0.7143 after user 1 leaves. After distillation, the accuracy of the new model is 0.7200. For CIFAR-10 using a Vision Transformer model, the original global model achieves an accuracy of 0.9572 before user 1 leaves and 0.9176 after user 1 leaves. After distillation, the accuracy of the new model is 0.9279. This improvement may be attributed to the increased diversity and complexity of the images in CINIC-10, which are partially sourced from ImageNet. Such characteristics can provide richer feature representations during distillation, potentially enhancing the model’s generalization ability.

No matter what dataset we use, the new model’s accuracy without any key trigger is always similar to the accuracy of the old model predicting with user 1’s key trigger.

As expected, the new global model’s accuracy is similar to the old model’s accuracy with user 1’s key trigger. In other words, the accuracy of $P_{M_{new}}(X_i)$ is closely related to the accuracy of $P_M(X_i^K)$. This is reasonable because, as the equation 10 means, we want the new global model to learn as much as possible from the old model with the key trigger of user 1, making $P_{M_{new}}(X_i)$ close to $P_M(X_i^K)$ or rather Y_i . Accordingly, we can infer that the new model M_{new} can forget user1’s data without any key trigger.

To further verify that the model distilled using CINIC-10 achieves effective unlearning, we compute the Jensen–Shannon (JS) divergence on the test dataset. The JS divergence between the distilled model and the retrained model

(trained using data from users 2 to 5) is 0.0191, indicating a high degree of similarity and suggesting successful unlearning. In addition, the JS divergence between the distilled model with user 1’s key trigger and without the key trigger is 0.0067. This extremely small divergence indicates that the model produces nearly identical outputs regardless of the presence of the key trigger.

Though the accuracy of the model drops after key trigger fusing distillation, the accuracy can be improved in the following rounds of the train with the remaining user’s contribution. This method (post-training) is also used in Federated Unlearning with Knowledge Distillation [42] to improve the accuracy of the model after unlearning. We verify post-training performance in the CIFAR-10 experiment with a ResNet18 model. The test accuracy after distillation is 0.7200. After applying post-training using data from users 2 to 5, the test accuracy improves to 0.8412. We also find that for CIFAR-10 with ViT, the accuracy is better preserved, suggesting that adopting an appropriate model enables effective unlearning while maintaining overall model performance.

5. Discussion

5.1. Trusted Revocation Center Assumption

As described in section 3, in our proposed scheme, the revocation center, which is in charge of assigning user-specific trapdoors in the registration phase and processing legitimate unlearning requests in the inference phase, is assumed to be a fully trusted entity. We think the assumption is reasonable since it is just like a key generation center (KGC) in other encryption schemes. We acknowledge that the assumption of a fully trusted revocation center is a strong one. Therefore, we explicitly analyze the security implications and limitations of the proposed scheme where this assumption does not hold.

If the revocation center is not trusted, the following security threats to the entire system will occur.

1. **Revocation Integrity:** An adversary who gains control of the revocation center can determine which trapdoors are triggered. As a result, unauthorized unlearning requests can be issued, or authorized requests can be ignored, causing legitimate data or model knowledge to be disabled without the consent of the corresponding users. Therefore, the integrity

of the revocation process can no longer be guaranteed.

2. **Trapdoor Leakage:** Once the revocation center is compromised, the user trapdoors will be leaked to the adversary. The adversary can then train the data with the target’s trapdoor embedded, reducing the effectiveness of data revocation.
3. **Single Point of Failure:** The revocation center serves as the sole trusted authority responsible for generating revocation trapdoors and processing revocation-related operations. Consequently, it constitutes a single point of failure within the proposed framework. Once this entity becomes unavailable, the entire revocation mechanism may cease to function correctly.

We acknowledge that the whole system relies on the revocation center in the current design. This is for simplifying trapdoor management and revocation authorization. Although the proposed scheme remains secure under the assumption of an honest and uncompromised revocation center, compromising this trusted authority may result in system-wide security and availability consequences. Investigating distributed or threshold-based revocation authorities to mitigate this limitation is an important direction for future work.

5.2.

6. Conclusion and Future Work

In this work, we propose a series of steps to turn off the influence of a certain user on the model. After a user leaves the service system, the user’s corresponding key trigger is added to the queried data before each prediction. Thus, the model predicts that the leaving user never provides any data to train the model. Furthermore, after a certain number of users leave, the distill center updates the global model. As a result, without any key trigger, the new global model can still forget the influence of the users who have left.

Although MUTED has solved some problems, such as the emergence of noise or the limited space for keys, there is still room for improvement. At present, we refer to a static trapdoor attack and develop the user’s key trigger. In our future work, we will try to use a dynamic trapdoor attack to calculate the triggers for more flexibility.

Moreover, the accuracy of the global model drops after key trigger fusing distillation because the model forgets about the users who have left. In order to enhance the performance of the global model after key distillation, we can continuously train the global model along with the remaining user model for several turns. Moreover, since machine learning trapdoor attacks can also be launched on different types of media, such as texts [43] and videos [44], we will extend our experiments to other media and see the unlearning effects.

ACKNOWLEDGMENT

This work was supported by the National Science and Technology Council, Taiwan, under the Grant NSTC 113-2634-F-004-001-MBK.

References

- [1] K. P. Bennett, E. Parrado-Hernández, The interplay of optimization and machine learning research, *J. Mach. Learn. Res.* 7 (2006) 1265–1281.
- [2] J. Nichols, H. Chan, M. Baker, Machine learning: applications of artificial intelligence to imaging and diagnosis, *Biophysical Reviews* 11 (09 2018). doi: 10.1007/s12551-018-0449-9.
- [3] I. H. Sarker, Machine learning: Algorithms, real-world applications and research directions, *SN Computer Science* 2 (3) (2021) 160. doi: 10.1007/s42979-021-00592-x.
- [4] C. Song, T. Ristenpart, V. Shmatikov, Machine learning models that remember too much, in: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS '17*, Association for Computing Machinery, New York, NY, USA, 2017, p. 587–601. doi: 10.1145/3133956.3134077.
- [5] C. J. Hoofnagle, B. Van Der Sloot, F. Z. Borgesius, The european union general data protection regulation: what it is and what it means, *Information & Communications Technology Law* 28 (1) (2019) 65–98.
- [6] J. S. Baik, Data privacy against innovation or against discrimination?: The case of the california consumer privacy act (ccpa), *Telematics and Informatics* 52 (2020) 101431. doi: <https://doi.org/10.1016/j.tele.2020.101431>.
- [7] H. Iwase, Overview of the act on the protection of personal information, *Eur. Data Prot. L. Rev.* 5 (2019) 92.
- [8] J. Beardwood, The new canadian consumer privacy protection act: A compliance briefing for privacy officers — a brief overview, *Computer Law Review International* 22 (1) (2021) 1–8 [cited 2024-04-17]. doi: 10.9785/crl-2021-220102.
- [9] L. Bourtole, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, N. Papernot, Machine unlearning, in: *2021 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2021. doi: 10.1109/sp40001.2021.00019.
- [10] J. Li, S. Ghosh, Random relabeling for efficient machine unlearning, *arXiv preprint arXiv:2305.12320* (2023).
- [11] I. Cho, C. Park, J. Hockenmaier, Vit-mul: A baseline study on recent machine unlearning methods applied to vision transformers, *arXiv preprint arXiv:2403.09681* (2024).
- [12] Z. Huang, X. Cheng, J. Zheng, H. Wang, Z. He, T. Li, X. Huang, Unified gradient-based machine unlearning with remain geometry enhancement, in: A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, C. Zhang (Eds.), *Advances in Neural Information Processing Systems*, Vol. 37, Curran Associates, Inc., 2024, pp. 26377–26414.
- [13] E. Chien, H. Wang, Z. Chen, P. Li, Certified machine unlearning via noisy stochastic gradient descent, in: A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, C. Zhang (Eds.), *Advances in Neural Information Processing Systems*, Vol. 37, Curran Associates, Inc., 2024, pp. 38852–38887.

- [14] V. S. Chundawat, A. K. Tarun, M. Mandal, M. Kankanhalli, Can bad teaching induce forgetting? unlearning in deep networks using an incompetent teacher, in: Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI'23/IAAI'23/EAAI'23, AAAI Press, 2023. doi:10.1609/aaai.v37i6.25879.
- [15] A. Nilsson, S. Smith, G. Ulm, E. Gustavsson, M. Jirstrand, A performance evaluation of federated learning algorithms, in: Proceedings of the Second Workshop on Distributed Infrastructures for Deep Learning, DIDL '18, Association for Computing Machinery, New York, NY, USA, 2018, p. 1–8. doi:10.1145/3286490.3286559.
- [16] L. Li, Y. Fan, M. Tse, K.-Y. Lin, A review of applications in federated learning, Computers & Industrial Engineering 149 (2020) 106854. doi:https://doi.org/10.1016/j.cie.2020.106854.
- [17] C. Zhang, Y. Xie, H. Bai, B. Yu, W. Li, Y. Gao, A survey on federated learning, Knowledge-Based Systems 216 (2021) 106775. doi:https://doi.org/10.1016/j.knosys.2021.106775.
- [18] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, T. Parcollet, N. D. Lane, Flower: A friendly federated learning research framework, CoRR abs/2007.14390 (2020). arXiv:2007.14390.
- [19] H. Xu, T. Zhu, L. Zhang, W. Zhou, P. S. Yu, Machine unlearning: A survey, ACM Computing Surveys 56 (1) (2023) 1–36. doi:10.1145/3603620.
- [20] J. Xu, Z. Wu, C. Wang, X. Jia, Machine unlearning: Solutions and challenges, IEEE Transactions on Emerging Topics in Computational Intelligence (2024) 1–19doi:10.1109/tetci.2024.3379240.
- [21] Y. Zhang, Z. Lu, F. Zhang, H. Wang, S. Li, Machine unlearning by reversing the continual learning, Applied Sciences 13 (16) (2023). doi:10.3390/app13169341.
- [22] C. Guo, T. Goldstein, A. Hannun, L. van der Maaten, Certified data removal from machine learning models (2019). arXiv:1911.03030.
- [23] A. Dhasade, Y. Ding, S. Guo, A. marie Kermarrec, M. D. Vos, L. Wu, Quickdrop: Efficient federated unlearning by integrated dataset distillation (2023). arXiv:2311.15603.
- [24] Y. Li, C. Chen, X. Zheng, J. Zhang, Federated unlearning via active forgetting (2023). arXiv:2307.03363.
- [25] J. Ji, Y. Liu, Y. Zhang, G. Liu, R. R. Kompella, S. Liu, S. Chang, Reversing the forget-retain objectives: An efficient llm unlearning framework from logit difference, in: A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, C. Zhang (Eds.), Advances in Neural Information Processing Systems, Vol. 37, Curran Associates, Inc., 2024, pp. 12581–12611.
- [26] R. Xu, S. Hong, P. Chi, M. Wang, A revocation key-based approach towards efficient federated unlearning, in: 2023 18th Asia Joint Conference on Information Security (AsiaJCIS), IEEE Computer Society, Los Alamitos, CA, USA, 2023, pp. 17–24. doi:10.1109/AsiaJCIS60284.2023.00014.
- [27] A. Salem, R. Wen, M. Backes, S. Ma, Y. Zhang, Dynamic backdoor attacks against machine learning models, CoRR abs/2003.03675 (2020). arXiv:2003.03675.
- [28] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network (2015). arXiv:1503.02531.
- [29] J. Gou, B. Yu, S. J. Maybank, D. Tao, Knowledge distillation: A survey, International Journal of Computer Vision 129 (6) (2021) 1789–1819. doi:10.1007/s11263-021-01453-z.
- [30] Y. Tu, P. Hu, J. Ma, Towards reliable empirical machine unlearning evaluation: A cryptographic game perspective (2025). arXiv:2404.11577.
- [31] Y. Guo, Y. Zhao, S. Hou, C. Wang, X. Jia, Verifying in the dark: Verifiable machine unlearning by using invisible backdoor triggers, IEEE Transactions on Information Forensics and Security 19 (2024) 708–721. doi:10.1109/TIFS.2023.3328269.

- [32] J. Niu, P. Liu, X. Zhu, K. Shen, Y. Wang, H. Chi, Y. Shen, X. Jiang, J. Ma, Y. Zhang, A survey on membership inference attacks and defenses in machine learning, *Journal of Information and Intelligence* 2 (5) (2024) 404–454. doi:<https://doi.org/10.1016/j.jiixd.2024.02.001>.
- [33] J. Hayes, I. Shumailov, E. Triantafillou, A. Khalifa, N. Papernot, Inexact unlearning needs more careful evaluations to avoid a false sense of privacy (2024). arXiv:2403.01218.
- [34] J. Zhang, D. Das, G. Kamath, F. Tramer, Position: Membership Inference Attacks Cannot Prove That a Model was Trained on Your Data , in: 2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML), IEEE Computer Society, Los Alamitos, CA, USA, 2025, pp. 333–345. doi: 10.1109/SaTML64287.2025.00025.
- [35] J. Cheng, H. Amiri, Mu-bench: A multitask multimodal benchmark for machine unlearning, arXiv preprint arXiv:2406.14796 (2024).
- [36] S. Shen, C. Zhang, Y. Zhao, A. Bialkowski, W. Chen, M. Xu, Machine unlearning for streaming forgetting, in: ICLR 2025 Conference, 2024, under review / submitted.
- [37] M.-R. Amini, V. Feofanov, L. Pauletto, L. Hadjadj, E. Devijver, Y. Maximov, Self-training: A survey (2023). arXiv:2202.12040.
- [38] Q. Xie, T. Luong, E. Hovy, Q. V. Le, Self-training with noisy student improves imagenet classification, in: CVPR, 2020.
- [39] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778. doi:10.1109/CVPR.2016.90.
- [40] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby, An image is worth 16x16 words: Transformers for image recognition at scale, in: International Conference on Learning Representations, 2021.
- [41] L. N. Darlow, E. J. Crowley, A. Antoniou, A. J. Storkey, CINIC-10 is not imagenet or CIFAR-10, CoRR abs/1810.03505 (2018). arXiv:1810.03505.
- [42] C. Wu, S. Zhu, P. Mitra, Federated unlearning with knowledge distillation (2022). arXiv:2201.09441.
- [43] S. Sun, H. Sun, H. Zhang, Y. Zhang, A study of backdoor attacks on data distillation for text classification tasks, in: 2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), 2024, pp. 2517–2522. doi:10.1109/TrustCom63139.2024.00350.
- [44] J. Zhao, H. Yang, H. He, J. Peng, W. Zhang, J. Ni, A. K. Sangaiah, A. Castiglione, Backdoor two-stream video models on federated learning, ACM Trans. Multimedia Comput. Commun. Appl. 20 (11) (Sep. 2024). doi:10.1145/3651307.